CÓDIGOCLARO · MATERIAL DE ESTUDIO

PL/SQL: cursores, paquetes y procedimientos

Bloques Oracle, manejo de errores, transacciones y organización de lógica de datos

1. Bloques PL/SQL y alcance

Objetivo: Identificar las partes de un bloque anónimo y separar declaración, ejecución y gestión de excepciones.

PL/SQL amplía SQL con variables, control de flujo, procedimientos y excepciones. Un bloque anónimo puede iniciar con `DECLARE`, ejecuta sentencias después de `BEGIN` y puede gestionar errores en `EXCEPTION`; la sección de declaración y la de excepciones son opcionales, pero `BEGIN` y `END` delimitan el cuerpo. Una barra `/` suele ser una orden del cliente para enviar el bloque, no parte del lenguaje PL/SQL. Declara variables con tipos claros, califica referencias ambiguas y evita efectos laterales inesperados. Para depurar, reduce el bloque a una operación comprobable y registra valores seguros, sin credenciales ni datos privados.

Código · ejemplo

```
[PL/SQL]
DECLARE
  v_total NUMBER := 0;
BEGIN
  v_total := 2 + 3;
  DBMS_OUTPUT.PUT_LINE(v_total);
END;
/
```

Práctica: ¿Qué partes son del bloque y cuál es la barra final en SQL*Plus o SQLcl?

Respuesta razonada: `DECLARE`, `BEGIN`, las sentencias, `END` y el punto y coma forman el bloque. `/` es una instrucción del cliente que ejecuta el contenido del búfer; no se envía como una sentencia PL/SQL al servidor. Una herramienta distinta puede tener otra forma de ejecutar bloques.

2. Tipos anclados y SELECT INTO

Objetivo: Declarar variables que sigan el tipo de columnas y recuperar exactamente una fila de forma explícita.

El atributo `%TYPE` ancla una variable al tipo de una columna, y `%ROWTYPE` crea un registro con la estructura de una fila. Esto reduce divergencias cuando cambia el esquema. `SELECT ... INTO` en PL/SQL espera una sola fila: cero filas causan `NO_DATA_FOUND` y más de una, `TOO_MANY_ROWS`. Si el resultado puede tener varias filas, utiliza un cursor o una colección. Las columnas y variables con nombres parecidos conviene calificarlas con alias; así una condición no compara accidentalmente una columna consigo misma. La cantidad y orden de expresiones seleccionadas deben concordar con las variables de destino.

Código · ejemplo

```
[PL/SQL]
DECLARE
  v_nombre employees.last_name%TYPE;
BEGIN
  SELECT last_name INTO v_nombre
  FROM employees WHERE employee_id = 100;
END;
/
```

Práctica: ¿Qué cambia si `employee_id` no es único en la tabla consultada?

Respuesta razonada: El `SELECT INTO` lanza `TOO_MANY_ROWS`; no elige una fila arbitraria. Si el negocio exige una única persona, protege la clave con una restricción única y trata la excepción como anomalía. Si se esperan varias personas, modela una consulta de varias filas en vez de ocultar el error.

3. Excepciones y recuperación

Objetivo: Distinguir condiciones esperadas de errores inesperados y devolver diagnósticos que permitan actuar.

PL/SQL propaga excepciones hasta un manejador compatible si no las captura en el bloque actual. Se pueden manejar excepciones predefinidas, declarar otras propias o asociar códigos mediante `PRAGMA EXCEPTION_INIT`. Un manejador `WHEN OTHERS` debe ser el último y no debería tragarse el fallo; registra contexto permitido y vuelve a lanzar cuando no exista una recuperación válida. `SQLCODE` y `SQLERRM` ayudan al diagnóstico, pero no sustituyen un contrato de errores. Divide el trabajo en bloques pequeños si una operación requiere tratamiento distinto. Recuerda que capturar una excepción no revierte automáticamente todas las operaciones previas: razona por separado sobre transacciones y estado persistente.

Código · ejemplo

```
[PL/SQL]
BEGIN
  INSERT INTO audit_log(message) VALUES ('inicio');
EXCEPTION
  WHEN OTHERS THEN
    RAISE;
END;
/
```

Práctica: ¿Por qué un `WHEN OTHERS THEN NULL` puede ocultar un fallo grave?

Respuesta razonada: El cliente recibe una terminación aparentemente normal aunque la operación no haya cumplido su propósito, y puede continuar con estado parcial. Si no puedes reparar el error, propágalo o tradúcelo a una excepción documentada conservando el contexto. Si escribes una traza, evita registrar datos sensibles y define su propia transacción.

4. Cursores explícitos

Objetivo: Recorrer varias filas con el ciclo de vida de un cursor y evitar lecturas incompletas o recursos abiertos.

Un cursor explícito define una consulta de varias filas. El ciclo clásico abre el cursor, recupera filas con `FETCH` y verifica `SQL%NOTFOUND` o el atributo correspondiente del cursor antes de usar datos no asignados. Al terminar, se cierra. Si hay una excepción entre `OPEN` y `CLOSE`, una estructura de limpieza debe asegurar el cierre. Los atributos `%FOUND`, `%NOTFOUND`, `%ROWCOUNT` e `%ISOPEN` describen el estado actual del cursor; su significado depende de cuál cursor se consulta. Para recorridos simples, un cursor FOR LOOP abre, recupera y cierra automáticamente, y suele ser más seguro y legible.

Código · ejemplo

```
[PL/SQL]
DECLARE
  CURSOR c_emp IS SELECT employee_id, last_name FROM employees;
  v_id employees.employee_id%TYPE;
  v_name employees.last_name%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_id, v_name;
    EXIT WHEN c_emp%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_id || ': ' || v_name);
  END LOOP;
  CLOSE c_emp;
END;
/
```

Práctica: ¿Por qué se comprueba `%NOTFOUND` inmediatamente tras `FETCH`?

Respuesta razonada: Tras `FETCH`, si no había más filas, no se debe procesar el contenido de las variables como si fuese una fila nueva. La condición de salida evita una lectura fantasma o datos anteriores. En producción, un cursor FOR LOOP elimina la gestión manual del cierre cuando no se necesita acceso explícito.

5. Cursores parametrizados y FOR LOOP

Objetivo: Reutilizar una consulta con parámetros y aprovechar la gestión automática del cursor cuando el recorrido es sencillo.

Un cursor parametrizado permite especificar valores de filtro al abrirlo; evita duplicar consultas con criterios distintos. Los parámetros se declaran con tipos adecuados y se suministran en la invocación. Un cursor FOR LOOP declara implícitamente un registro para cada fila y gestiona apertura, recuperación y cierre. El registro solo vive en el alcance del bucle. Usa `ORDER BY` cuando el orden forme parte del resultado requerido, porque las filas SQL no tienen orden garantizado sin ordenación. No recorras fila por fila si una sentencia SQL de conjunto expresa claramente la misma operación; el cambio de contexto entre SQL y PL/SQL tiene coste.

Código · ejemplo

```
[PL/SQL]
DECLARE
  CURSOR c_dept(p_dept NUMBER) IS
    SELECT employee_id FROM employees WHERE department_id = p_dept;
BEGIN
  FOR r IN c_dept(50) LOOP
    DBMS_OUTPUT.PUT_LINE(r.employee_id);
  END LOOP;
END;
/
```

Práctica: ¿Qué ocurre con el cursor FOR LOOP si la consulta no devuelve filas?

Respuesta razonada: El cuerpo del bucle ejecuta cero iteraciones, sin lanzar `NO_DATA_FOUND` solo por estar vacío. Si el negocio requiere que haya al menos una fila, comprueba ese requisito explícitamente. El parámetro limita la consulta y el cursor se cierra al salir normalmente o por excepción.

6. Paquetes: especificación y cuerpo

Objetivo: Separar el contrato público del detalle privado al agrupar procedimientos, funciones y datos relacionados.

Un paquete PL/SQL tiene una especificación pública y un cuerpo de implementación. La especificación declara lo que otros bloques pueden invocar; constantes, tipos, cursores y subprogramas que solo usa el paquete pueden permanecer privados en el cuerpo. Mantener pequeñas las interfaces reduce acoplamiento y permite cambiar la implementación conservando el contrato. Variables globales de paquete pueden persistir durante una sesión, por lo que no deben usarse como almacenamiento accidental entre solicitudes. Comprueba dependencias y recompila objetos afectados tras cambios de firma. Nombra parámetros y documenta si una rutina confirma transacciones o deja esa decisión al llamador.

Código · ejemplo

```
[PL/SQL]
CREATE OR REPLACE PACKAGE order_api AS
  PROCEDURE close_order(p_order_id IN orders.order_id%TYPE);
END order_api;
/
CREATE OR REPLACE PACKAGE BODY order_api AS
  PROCEDURE close_order(p_order_id IN orders.order_id%TYPE) IS
  BEGIN
    UPDATE orders SET status = 'CLOSED' WHERE order_id = p_order_id;
  END;
END order_api;
/
```

Práctica: ¿Por qué no se declara `close_order` solo en el cuerpo si lo usarán otros esquemas?

Respuesta razonada: Una rutina solo en el cuerpo es privada a los clientes del paquete. Para que sea invocable externamente, se declara en la especificación y luego se implementa en el cuerpo. También se deben conceder privilegios apropiados al consumidor; que la firma sea pública no significa que cualquier usuario pueda ejecutarla.

7. Procedimientos, funciones y modos

Objetivo: Elegir entre procedimiento y función y usar parámetros IN, OUT o IN OUT con un contrato claro.

Un procedimiento ejecuta una operación y puede devolver resultados por parámetros `OUT`; una función produce un valor y puede ser invocada en expresiones cuando cumple las restricciones del contexto. `IN` es el modo predeterminado, `OUT` devuelve un valor y `IN OUT` recibe y modifica uno. Prefiere funciones sin efectos laterales para cálculos y procedimientos para operaciones. El nombre de una rutina debe reflejar su resultado y el contrato debe aclarar qué ocurre con entradas inválidas, ausencia de datos y transacciones. Una función llamada desde SQL tiene restricciones sobre operaciones que modifican estado; no asumas que toda llamada PL/SQL sirve dentro de una consulta.

Código · ejemplo

```
[PL/SQL]
CREATE OR REPLACE FUNCTION tax_amount(p_base IN NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN ROUND(p_base * 0.21, 2);
END;
/
```

Práctica: ¿Qué contrato adicional definirías antes de usar la función con bases negativas o nulas?

Respuesta razonada: Define si `NULL` se propaga, se rechaza o se trata como cero, y si una base negativa es válida por devoluciones. La función implementa una operación financiera, así que tasa, moneda, escala y redondeo requieren reglas del dominio. No basta con que compile ni con el ejemplo de una entrada positiva.

8. Triggers y reglas de integridad

Objetivo: Comprender cuándo se ejecuta un trigger y escoger restricciones declarativas cuando expresan mejor la regla.

Un trigger puede ejecutarse antes o después de una sentencia o por fila, según su definición. Puede auditar, completar valores o aplicar reglas locales, pero la lógica oculta dificulta entender efectos de una escritura. Una restricción `NOT NULL`, `CHECK`, `UNIQUE` o clave foránea expresa integridad de forma declarativa y suele ser preferible cuando basta. Los triggers por fila pueden sufrir errores de tabla mutante si consultan la misma tabla que cambia, y los triggers encadenados complican el orden. Identifica el usuario y operación que activan el trigger, el volumen de filas y la transacción a la que pertenece. Prueba inserción, actualización, borrado y reversión.

Código · ejemplo

```
[PL/SQL]
CREATE OR REPLACE TRIGGER orders_bi
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
  :NEW.created_at := COALESCE(:NEW.created_at, SYSTIMESTAMP);
END;
/
```

Práctica: ¿Por qué una fecha predeterminada puede declararse como `DEFAULT` en vez de trigger?

Respuesta razonada: Una cláusula `DEFAULT` suele hacer más visible la regla y elimina código de trigger cuando solo se trata de completar un valor ausente. El trigger puede ser necesario si hay lógica contextual adicional. Comprueba la semántica de valores nulos, permisos y compatibilidad del esquema antes de elegir.

9. Transacciones y operaciones por conjuntos

Objetivo: Mantener atomicidad bajo control del llamador y preferir SQL de conjunto frente a bucles costosos.

Los cambios SQL forman parte de la transacción de la sesión hasta que se confirma o revierte según la gestión del cliente. Una rutina reutilizable normalmente no debería ejecutar `COMMIT` sin que el contrato lo exija, porque podría confirmar trabajo previo del llamador. `SAVEPOINT` permite establecer un punto de recuperación dentro de la transacción. Siempre que se pueda, una sentencia `UPDATE` o `INSERT ... SELECT` modifica un conjunto completo con semántica atómica y suele ser más eficiente que abrir un cursor y ejecutar una operación por fila. Para operaciones masivas, `BULK COLLECT` y `FORALL` reducen cambios de contexto, pero requieren límites de memoria y manejo cuidadoso de errores.

Código · ejemplo

```
[PL/SQL]
UPDATE employees
SET salary = salary * 1.03
WHERE department_id = 50;
-- La aplicación decide COMMIT o ROLLBACK
```

Práctica: ¿Qué ventaja aporta dejar `COMMIT` al código que coordina la transacción?

Respuesta razonada: El llamador puede combinar varias operaciones y confirmarlas o revertirlas como una unidad. Si una rutina interna confirma por su cuenta, la aplicación pierde esa posibilidad y puede persistir un estado incompleto. El contrato debe indicar claramente quién controla la transacción y qué fallos producen rollback.

10. Rendimiento, pruebas y despliegue

Objetivo: Verificar rutinas PL/SQL con datos límite, medir consultas y promover cambios de esquema con control.

Prueba las rutinas en una transacción controlada con entradas válidas, valores nulos, claves inexistentes, duplicados y errores de permisos. Comprueba tanto el resultado como el estado tras `ROLLBACK`. Para diagnosticar lento, identifica primero la sentencia SQL real y su plan; un cursor no es lento por definición, pero un bucle fila a fila suele ser señal de que una operación de conjunto es posible. Añade índices solo con un patrón de consulta medido. El despliegue debe versionar paquetes, tablas, grants y scripts de migración de manera coherente. No publiques credenciales en logs. Registra versión de Oracle probada y comportamiento esperado.

Código · ejemplo

```
[PL/SQL]
SAVEPOINT before_change;
BEGIN
  UPDATE orders SET status = 'CLOSED' WHERE order_id = 404;
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Pedido inexistente');
  END IF;
END;
/
```

Práctica: ¿Por qué comprobar `SQL%ROWCOUNT` y probar después un rollback?

Respuesta razonada: La sentencia válida puede afectar cero filas sin generar excepción; comprobar el contador convierte ese caso en un resultado explícito. Un rollback verifica que el bloque no dejó persistencia accidental en la prueba. En una aplicación, el identificador de pedido y los permisos deben validarse dentro del flujo transaccional.

Fuentes oficiales

Oracle AI Database 26ai · PL/SQL Language Reference · Oracle AI Database 26ai · SQL Language Reference · Oracle AI Database 26ai · PL/SQL Packages and Types Reference